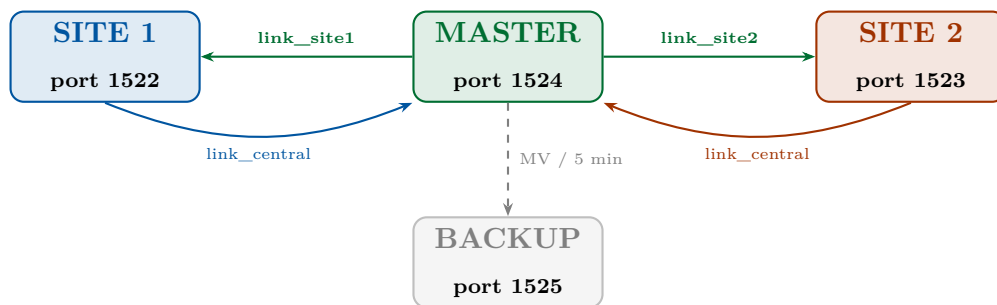

Université Hassan 1^{er}
Faculté des Sciences et Techniques de Settat
Département Génie Informatique

Rapport de Projet – Module : Bases de Données Distribuées

**Conception et Implémentation d'une
Base de Données Distribuée
pour une Plateforme e-Commerce B2B**



Technologie	Oracle XE 21c – 4 conteneurs Docker
Scénario 1	$R_1 : \text{Cat50} \ \& \ \text{qté} > 100 \rightarrow \text{Site 1}$ $R_2 : \text{Cat35} \ \& \ \text{qté} > 50 \rightarrow \text{Site 2}$
Scénario 2	$R_1 : \text{qté} \geq 100 \rightarrow \text{Site 1}$ $R_2 : \text{qté} < 100 \rightarrow \text{Site 2}$
Synchronisation	Triggers multi-maîtres + package anti-boucle
Séquences	Décalage modulo 3 (zéro collision inter-nœuds)
Sauvegarde	Vues matérialisées REFRESH COMPLETE / 5 min

Réalisé par :	Hajar Elhilali Yasmine Koulam
Filière :	Cycle Ingénieur – Génie Informatique
Niveau :	2 ^{ème} année
Année univ. :	2025 – 2026

Résumé

Ce rapport présente la conception et l'implémentation d'une base de données distribuée pour la plateforme e-commerce B2B **EShop**, déployée sur Oracle XE 21c dans quatre conteneurs Docker.

Le projet est organisé autour de deux scénarios de fragmentation horizontale de la table `LigneCommandes` :

- **Scénario 1** : fragmentation *dérivée* basée sur deux critères – la catégorie du produit (`idcateg`) et la quantité commandée. Les lignes électronique/ $qté > 100$ vont sur Site 1 et les lignes mobilier/ $qté > 50$ vont sur Site 2.
- **Scénario 2** : fragmentation *primaire* basée uniquement sur un seuil de quantité. Toutes les lignes avec $qté \geq 100$ vont sur Site 1, toutes celles avec $qté < 100$ vont sur Site 2. La partition est *complète* : chaque ligne appartient à exactement un fragment.

Les mécanismes implémentés incluent : l'initialisation par CTAS via DB Links, la réplication multi-maître par triggers compound, la gestion des identifiants distribués par séquences à décalage modulo 3, l'optimisation des requêtes par index couvrant, et la sauvegarde par vues matérialisées.

Mots-clés : Base de données distribuée, Oracle XE, Fragmentation horizontale, PL/SQL, Triggers, Séquences, Docker, DB Links.

Table des matières

Résumé	1
1 Introduction et Contexte du Projet	4
1.1 Problématique	4
1.2 Objectifs du projet	4
1.3 Architecture globale	4
1.4 Comparaison des deux scénarios	5
2 Infrastructure Technique	6
2.1 Déploiement Docker Compose	6
2.2 Résolution TNS	7
2.3 Database Links privés	7
2.4 Compte connecteur à moindre privilège	8
2.5 Synonymes : transparence de localisation	8
2.5.1 Synonymes publics du Master vers les sites	8
2.5.2 Synonymes publics des sites vers le Master	9
2.5.3 Synonymes privés du compte connecteur	9
3 Schéma Global et Gestion des Séquences	10
3.1 Schéma de la base globale	10
3.2 Stratégie des séquences – décalage modulo 3	10
3.2.1 Problématique des collisions d’identifiants	10
3.2.2 Solution : voies indépendantes	11
3.2.3 Procédure make_seq – point de départ dynamique	11
3.3 Jeu de données	12
4 Scénario 1 – Fragmentation par Catégorie et Quantité	13
4.1 Règles de fragmentation	13
4.2 Initialisation par CTAS	13
4.3 Triggers de distribution – Master	13
4.4 Triggers compound sur Site 1	14
4.5 Triggers BEFORE – Prévention des violations de clé étrangère	15

5	Scénario 2 – Fragmentation par Seuil de Quantité	17
5.1	Règles de fragmentation	17
5.2	Différences clés avec le Scénario 1	17
5.3	Initialisation par CTAS	18
5.4	Triggers de distribution – Master (Scénario 2)	18
5.5	Triggers compound sur Site 2 (Scénario 2)	19
6	Mécanismes de Synchronisation Communs	21
6.1	Package anti-boucle <code>pkg_synchro</code>	21
6.2	Journalisation autonome des erreurs	21
6.3	Procédures stockées PL/SQL	22
6.4	Sauvegarde par vues matérialisées	23
7	Optimisation des Requêtes	24
7.1	Q5 – Nombre de commandes par client (analyse du plan)	24
7.2	Q6 – Chiffre d'affaires distribué (UNION ALL)	25
8	Tests et Résultats	26
8.1	Méthodologie de test	26
8.2	Tests – Scénario 1	26
8.2.1	T1 – INSERT Cat50, qté=150 → Site 1	26
8.2.2	T2 – INSERT Cat35, qté=80 → Site 2	27
8.2.3	T3 – UPDATE : migration Site 2 → Site 1 (changement de catégorie)	28
8.2.4	T4 – Modification depuis Site 2 vers Site 1 (inter-sites)	29
8.2.5	T5 – Propagation de la suppression (DELETE)	30
8.3	Tests – Scénario 2	31
8.3.1	T1 – INSERT qté=150 → Site 1 (toute catégorie)	31
8.3.2	T2 – INSERT qté=40 → Site 2	32
8.3.3	T3 – UPDATE depuis Site 2 : migration Site 2 → Site 1	33
8.3.4	T4 – UPDATE : migration Site 1 → Site 2	34
8.3.5	T5 – Vérification de la partition complète	35
8.4	Q5 – Plan d'exécution avant et après index	36
8.5	Q6 – Requête distribuée : CA par catégorie	37
	Conclusion	38

1. Introduction et Contexte du Projet

1.1 Problématique

La plateforme **EShop** est un système de commerce électronique B2B marocain regroupant des entreprises de secteurs variés. Avec la croissance du volume de commandes, la centralisation de la table **LigneCommandes** sur un seul serveur pose des problèmes de performance et de disponibilité.

La solution retenue est une **fragmentation horizontale** : les données sont réparties sur plusieurs sites géographiques, chacun stockant un sous-ensemble (fragment) des lignes de commandes selon un prédicat métier. Les sites doivent rester synchronisés en temps réel et pouvoir fonctionner de manière semi-autonome.

1.2 Objectifs du projet

- Concevoir et implémenter deux scénarios de fragmentation horizontale sur Oracle XE 21c.
- Développer des **procédures stockées PL/SQL** pour toutes les opérations d'écriture (INSERT, UPDATE, DELETE).
- Garantir la **cohérence multi-maître** par des triggers de synchronisation avec mécanisme anti-boucle.
- Assurer l'**unicité des identifiants** distribués par un schéma de séquences à décalage modulo 3.
- Optimiser les requêtes de lecture (Q5) et calculer le CA distribué via DB Links (Q6).
- Mettre en place une **base de secours** par vues matérialisées.

1.3 Architecture globale

L'infrastructure comprend quatre nœuds Oracle XE 21c déployés dans des conteneurs Docker reliés par le réseau **eshop-net**.

Nœud	Conteneur	Port	Schéma	Rôle
Master	eshop-master	1524	eshop_admin	Base globale, seed et routage
Site 1	eshop-site1	1522	eshop_site1	Fragment R1
Site 2	eshop-site2	1523	eshop_site2	Fragment R2
Backup	eshop-backup	1525	eshop_backup	Vues matérialisées

Le **Master** démarre en premier et contient l'intégralité des données. Les sites attendent sa disponibilité (`service_healthy`) avant d'initier leur CTAS (CREATE TABLE AS SELECT).

1.4 Comparaison des deux scénarios

Critère	Scénario 1	Scénario 2
Prédicat R_1	idcateg=50 ET quantité>100	quantité≥100
Prédicat R_2	idcateg=35 ET quantité>50	quantité<100
Type	Fragmentation dérivée	Fragmentation primaire
Couverture	Partielle (cat 10 = master seul)	Totale (partition complète)
Produits Site 1	Cat 50 uniquement	Toutes catégories
Produits Site 2	Cat 35 uniquement	Toutes catégories
Critères trigger	Catégorie + quantité	Quantité seule

2. Infrastructure Technique

2.1 Déploiement Docker Compose

Chaque nœud est construit à partir d'un Dockerfile dédié. Les scripts d'initialisation sont montés dans `/container-entrypoint-initdb.d` et exécutés automatiquement au premier démarrage.

docker-compose.yml – extrait représentatif

```
1 services:
2   db-master:
3     container_name: eshop-master
4     ports: ["1524:1521"]
5     volumes:
6       - ./scripts/scenario1/master:/container-entrypoint-initdb.d
7       - ./tnsnames.ora:/opt/oracle/network/admin/tnsnames.ora
8       - ./scripts/scenario1/global-queries:/queries:ro
9     networks: [eshop-net]
10    healthcheck:
11      test: ["CMD", "healthcheck.sh"]
12      interval: 30s
13      retries: 5
14
15    db-site1:
16      depends_on:
17        db-master: { condition: service_healthy }
18      ports: ["1522:1521"]
19      volumes:
20        - ./scripts/scenario1/site1:/container-entrypoint-initdb.d
21
22    db-site2: # port 1523, scripts/scenario1/site2
23    db-backup: # port 1525, scripts/scenario1/backup
```

Deux fichiers Docker Compose

Un fichier `docker-compose.yml` est dédié au Scénario 1 et un fichier `docker-compose.scenario2.yml` au Scénario 2. Pour basculer d'un scénario à l'autre, il suffit d'utiliser `docker-compose down -v` puis de relancer avec le bon fichier. Les volumes sont réinitialisés pour repartir d'une base propre.

2.2 Résolution TNS

Un fichier `tnsnames.ora` unique est monté dans chaque conteneur via la variable d'environnement `TNS_ADMIN=/opt/oracle/network/admin`.

tnsnames.ora

```

1 BD-MASTER = (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)
2             (HOST=db-master)(PORT=1521))
3             (CONNECT_DATA=(SERVICE_NAME=XEPDB1)))
4
5 BD-SITE1   = (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)
6             (HOST=db-site1)(PORT=1521))
7             (CONNECT_DATA=(SERVICE_NAME=XEPDB1)))
8
9 BD-SITE2   = (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)
10            (HOST=db-site2)(PORT=1521))
11            (CONNECT_DATA=(SERVICE_NAME=XEPDB1)))

```

2.3 Database Links privés

Les DB Links sont **privés** (propriété du schéma créateur, non empruntables par d'autres utilisateurs). Chaque nœud maintient les liens nécessaires à ses opérations.

Nœud	Lien	Destination	Usage
Master	link_site1	Site 1	Propagation INSERT/UPDATE/DELETE
Master	link_site2	Site 2	Propagation INSERT/UPDATE/DELETE
Site 1	link_central	Master	CTAS, rapatriement parents, synchro
Site 1	link_site2	Site 2	Routage croisé
Site 2	link_central	Master	CTAS, rapatriement parents, synchro
Site 2	link_site1	Site 1	Routage croisé
Backup	link_central	Master	Rafraîchissement vues matérialisées

Création des DB Links – Master

```

1 CONNECT eshop_admin/EshopPassword123@//localhost:1521/XEPDB1
2
3 CREATE DATABASE LINK link_site1
4     CONNECT TO eshop_link IDENTIFIED BY "LinkPwd123"
5     USING 'BD-SITE1';
6
7 CREATE DATABASE LINK link_site2
8     CONNECT TO eshop_link IDENTIFIED BY "LinkPwd123"

```

```
9 USING 'BD-SITE2';
```

2.4 Compte connecteur à moindre privilège

Un utilisateur `eshop_link` est créé sur chaque site avec uniquement les droits nécessaires (SELECT + EXECUTE) pour garantir le principe du moindre privilège.

Compte connecteur – Site 1

```
1 CREATE USER eshop_link IDENTIFIED BY "LinkPwd123";
2 GRANT CREATE SESSION TO eshop_link;
3 GRANT SELECT ON eshop_site1.lignecommandes1 TO eshop_link;
4 GRANT EXECUTE ON eshop_site1.INSERTligne TO eshop_link;
5 GRANT EXECUTE ON eshop_site1.DELETEligne TO eshop_link;
6 GRANT EXECUTE ON eshop_site1.updateligne TO eshop_link;
7 -- + synonymes prives pour eshop_link
```

2.5 Synonymes : transparence de localisation

Les **synonymes** jouent un rôle central dans l'architecture : ils masquent l'emplacement physique des objets et les liens de bases de données. Le code des triggers, des procédures et des requêtes manipule ainsi de simples noms de tables, sans répéter les clauses `@link_...` ni les schémas distants. Trois familles de synonymes coexistent.

2.5.1 Synonymes publics du Master vers les sites

Sur le **Master**, des synonymes publics pointent vers les fragments distants. Ils permettent aux requêtes distribuées (Q6) et aux outils d'administration d'interroger les sites avec un nom court (`lignecommandes1`, `produits2...`).

Synonymes publics – Master vers Site 1 et Site 2

```
1 -- Vers le fragment Site 1
2 CREATE OR REPLACE PUBLIC SYNONYM lignecommandes1 FOR lignecommandes1@link_site1
3 ;
4 CREATE OR REPLACE PUBLIC SYNONYM produits1 FOR produits1@link_site1;
5
6 -- Vers le fragment Site 2
7 CREATE OR REPLACE PUBLIC SYNONYM lignecommandes2 FOR lignecommandes2@link_site2
8 ;
9 CREATE OR REPLACE PUBLIC SYNONYM produits2 FOR produits2@link_site2;
10 CREATE OR REPLACE PUBLIC SYNONYM commandes2 FOR commandes2@link_site2;
```

Grâce à eux, la requête Q6 s'écrit `FROM lignecommandes1 ... UNION ALL ... FROM lignecommandes2` au lieu de `eshop_site1.lignecommandes1@link_site1`.

2.5.2 Synonymes publics des sites vers le Master

Sur chaque site, des synonymes publics pointent vers les tables *globales* du **Master**. Une écriture exécutée sur un site sur le nom `lignecommandes` (sans suffixe) est donc automatiquement dirigée vers la base globale — c'est ce qui rend le multi-maître transparent.

Synonymes publics – Site vers Master

```
1 CREATE OR REPLACE PUBLIC SYNONYM clients          FOR clients@link_central;
2 CREATE OR REPLACE PUBLIC SYNONYM produits         FOR produits@link_central;
3 CREATE OR REPLACE PUBLIC SYNONYM commandes        FOR commandes@link_central;
4 CREATE OR REPLACE PUBLIC SYNONYM lignecommandes    FOR lignecommandes@link_central
;
;
```

2.5.3 Synonymes privés du compte connecteur

Le compte `eshop_link` dispose de synonymes *privés* qui résolvent les appels distants **non qualifiés**. Quand le Master exécute `INSERTligne@link_site1` ou interroge `produits1`, la résolution se fait côté distant via ces synonymes, sans exposer le schéma propriétaire.

Synonymes privés – eshop_link (Site 2)

```
1 CREATE OR REPLACE SYNONYM eshop_link.produits2      FOR eshop_site2.produits2;
2 CREATE OR REPLACE SYNONYM eshop_link.commandes2    FOR eshop_site2.commandes2
;
3 CREATE OR REPLACE SYNONYM eshop_link.lignecommandes2 FOR eshop_site2.
  lignecommandes2;
4 CREATE OR REPLACE SYNONYM eshop_link.INSERTligne    FOR eshop_site2.
  INSERTligne;
5 CREATE OR REPLACE SYNONYM eshop_link.updateligne    FOR eshop_site2.
  updateligne;
6 CREATE OR REPLACE SYNONYM eshop_link.DELETEligne    FOR eshop_site2.
  DELETEligne;
```

Bénéfice : indépendance à la localisation

Si un fragment était déplacé sur un autre serveur, seule la définition du synonyme (ou du DB Link) serait à modifier : ni les triggers, ni les procédures, ni les requêtes métier ne changeraient. Les synonymes assurent la **transparence de localisation**, l'un des objectifs fondamentaux d'une base de données distribuée.

3. Schéma Global et Gestion des Séquences

3.1 Schéma de la base globale

La base **Master** contient quatre tables dont `lignecommandes` est la seule à être fragmentée horizontalement.

Schéma Master – tables principales

```
1 ALTER SESSION SET CURRENT_SCHEMA = eshop_admin;
2
3 CREATE TABLE clients (
4     idclient    NUMBER DEFAULT seq_clients.NEXTVAL PRIMARY KEY,
5     codeclient  VARCHAR2(50) NOT NULL,
6     societe     VARCHAR2(100) NOT NULL
7 );
8 CREATE TABLE produits (
9     idproduit   NUMBER DEFAULT seq_produits.NEXTVAL PRIMARY KEY,
10    designation  VARCHAR2(100) NOT NULL,
11    prixunitaire NUMBER(10,2) NOT NULL,
12    idcateg     NUMBER NOT NULL
13 );
14 CREATE TABLE commandes (
15     idcommande  NUMBER DEFAULT seq_commandes.NEXTVAL PRIMARY KEY,
16     idemploye   NUMBER,
17     idclient    NUMBER NOT NULL REFERENCES clients(idclient),
18     datecommande DATE NOT NULL
19 );
20 CREATE TABLE lignecommandes (
21     idlignecommande NUMBER DEFAULT seq_lignecommandes.NEXTVAL PRIMARY KEY,
22     idcommande   NUMBER NOT NULL REFERENCES commandes(idcommande),
23     idproduit    NUMBER NOT NULL REFERENCES produits(idproduit),
24     quantite     NUMBER NOT NULL,
25     remise       NUMBER(5,2)
26 );
```

3.2 Stratégie des séquences – décalage modulo 3

3.2.1 Problématique des collisions d'identifiants

Dans un système multi-maître, chaque nœud génère des identifiants de manière autonome. Sans coordination, des collisions de clés primaires surviennent inévitablement lors de la synchronisation entre nœuds.

3.2.2 Solution : voies indépendantes

Chaque nœud se voit attribuer une **voie** exclusive dans l'espace des entiers, définie par le reste de la division euclidienne par 3 :

Nœud	$\text{mod}(\text{ID}, 3)$	Exemples	Déclaration
Master	1	1, 4, 7, 10...	START WITH 1 INCREMENT BY 3
Site 1	2	2, 5, 8, 11...	START WITH s1 INCREMENT BY 3
Site 2	0	3, 6, 9, 12...	START WITH s2 INCREMENT BY 3

Garantie mathématique

Les ensembles $\{1, 4, 7, \dots\}$, $\{2, 5, 8, \dots\}$ et $\{3, 6, 9, \dots\}$ sont **strictement dis-joints** : aucune collision n'est possible, quelle que soit la durée de fonctionnement ou le volume de données inséré.

3.2.3 Procédure make_seq – point de départ dynamique

Sur les sites, les données importées par CTAS contiennent déjà des ID. La procédure `make_seq` calcule dynamiquement le premier identifiant libre dans la bonne voie :

Procédure make_seq (Site 1 – voie 2)

```

1 DECLARE
2     PROCEDURE make_seq(p_seq VARCHAR2, p_col VARCHAR2, p_tab VARCHAR2) IS
3         v_max NUMBER; v_start NUMBER;
4     BEGIN
5         EXECUTE IMMEDIATE
6             'SELECT NVL(MAX('||p_col||'),0) FROM '||p_tab INTO v_max;
7         v_start := v_max + 1;
8         WHILE MOD(v_start, 3) <> 2 LOOP    -- voie 2 pour Site 1
9             v_start := v_start + 1;      -- (<> 0 pour Site 2)
10        END LOOP;
11        EXECUTE IMMEDIATE 'CREATE SEQUENCE '||p_seq
12            ||' START WITH '||v_start||' INCREMENT BY 3 NOCACHE NOCYCLE';
13    END make_seq;
14 BEGIN
15     make_seq('seq_clients1',      'idclient',      'clients1');
16     make_seq('seq_produits1',     'idproduit',     'produits1');
17     make_seq('seq_commandes1',    'idcommande',  'commandes1');
18     make_seq('seq_lignecommandes1', 'idlignecommande', 'lignecommandes1');
19 END;
20 /

```

3.3 Jeu de données

Le jeu de données comprend **50 entreprises marocaines**, **30 produits**, **100 commandes** et **300 lignes de commandes**.

ID	Désignation	Cat	Prix (DH)	Site (Sc.1)	Site (Sc.2)
20001	Serveur Dell PowerEdge R750	50	18 500	Site 1	Site 1
20005	Firewall Fortinet FortiGate 200E	50	22 000	Site 1	Site 1
20011	Bureau direction chêne massif	35	3 200	Site 2	Dépend qté
20014	Table de conférence 12 places	35	4 800	Site 2	Dépend qté
20021	Ramette papier A4 80g	10	85	Master seul	Site 1

4. Scénario 1 – Fragmentation par Catégorie et Quantité

4.1 Règles de fragmentation

$$R_1 = \sigma_{idCatégorie=50 \wedge quantité>100}(LigneCommandes) \longrightarrow \text{Site 1}$$

$$R_2 = \sigma_{idCatégorie=35 \wedge quantité>50}(LigneCommandes) \longrightarrow \text{Site 2}$$

Remarque : les lignes ne satisfaisant ni R_1 ni R_2 (ex. cat 10) restent uniquement sur le Master.

4.2 Initialisation par CTAS

CTAS – Site 1 (R1) et Site 2 (R2)

```
1  -- Site 1 : R1 = idcateg=50 AND quantite > 100
2  CREATE TABLE lignecommandes1 AS
3      SELECT lc.idlignecommande, lc.idcommande, lc.idproduit,
4             lc.quantite, lc.remise
5      FROM   lignecommandes@link_central lc
6      JOIN   produits@link_central p ON p.idproduit = lc.idproduit
7      WHERE  p.idcateg = 50 AND lc.quantite > 100;
8
9  -- Site 2 : R2 = idcateg=35 AND quantite > 50
10 CREATE TABLE lignecommandes2 AS
11     SELECT lc.idlignecommande, lc.idcommande, lc.idproduit,
12            lc.quantite, lc.remise
13     FROM   lignecommandes@link_central lc
14     JOIN   produits@link_central p ON p.idproduit = lc.idproduit
15     WHERE  p.idcateg = 35 AND lc.quantite > 50;
```

4.3 Triggers de distribution – Master

Le trigger SYC_INSERT_LIGNE du Master route chaque INSERT vers le site approprié selon la catégorie du produit et la quantité.

SYC_INSERT_LIGNE et SYC_UPDATE_LIGNE – Master (Scénario 1)

```

1  -- INSERT : routage vers Site 1 ou Site 2
2  CREATE OR REPLACE TRIGGER SYC_INSERT_LIGNE
3  AFTER INSERT ON lignecommandes FOR EACH ROW
4  DECLARE
5      v_cat NUMBER;
6      NQ      lignecommandes.quantite%TYPE := :NEW.quantite;
7  BEGIN
8      IF pkg_synchro.is_replicating = TRUE THEN RETURN; END IF;
9      SELECT idcateg INTO v_cat FROM produits
10         WHERE idproduit = :NEW.idproduit;
11
12      IF (v_cat = 50 AND NQ > 100) THEN          -- R1 -> Site 1
13          EXECUTE IMMEDIATE
14              'BEGIN INSERTligne@link_site1(:1,:2,:3,:4,:5); END; '
15          USING :NEW.idlignecommande,:NEW.idcommande,
16              :NEW.idproduit,:NEW.quantite,:NEW.remise;
17      ELSIF (v_cat = 35 AND NQ > 50) THEN        -- R2 -> Site 2
18          EXECUTE IMMEDIATE
19              'BEGIN INSERTligne@link_site2(:1,:2,:3,:4,:5); END; '
20          USING :NEW.idlignecommande,:NEW.idcommande,
21              :NEW.idproduit,:NEW.quantite,:NEW.remise;
22      END IF;
23  END;
24  /

```

4.4 Triggers compound sur Site 1

Le **compound trigger** (AFTER EACH ROW + AFTER STATEMENT) évite l'erreur ORA-04091 (table en mutation) lors des suppressions en lot.

SYC_INSERT_LIGNE_SITE1 – Compound trigger

```

1  CREATE OR REPLACE TRIGGER SYC_INSERT_LIGNE_SITE1
2  FOR INSERT ON lignecommandes1
3  COMPOUND TRIGGER
4      TYPE t_id_list IS TABLE OF lignecommandes1.idlignecommande%TYPE;
5      v_ids_to_delete t_id_list := t_id_list();
6
7      AFTER EACH ROW IS
8          v_categ NUMBER;
9      BEGIN
10         IF pkg_synchro.is_replicating = FALSE OR
11            pkg_synchro.is_replicating IS NULL THEN
12             SELECT idcateg INTO v_categ FROM produits
13                WHERE idproduit = :NEW.idproduit;
14
15             -- 1. Propagation vers le Master (toujours)
16             EXECUTE IMMEDIATE
17                 'BEGIN INSERTligne@link_central(:1,:2,:3,:4,:5); END; '
18             USING :NEW.idlignecommande,:NEW.idcommande,

```



```

19         :NEW.idproduit, :NEW.quantite, :NEW.remise;
20
21         -- 2. Marquer pour suppression si ne respecte pas R1
22         IF NOT (v_categ = 50 AND :NEW.quantite > 100) THEN
23             v_ids_to_delete.EXTEND;
24             v_ids_to_delete(v_ids_to_delete.LAST) := :NEW.idlignecommande;
25         END IF;
26     END IF;
27 END AFTER EACH ROW;
28
29 AFTER STATEMENT IS
30 BEGIN
31     IF v_ids_to_delete.COUNT > 0 THEN
32         pkg_synchro.is_replicating := TRUE;
33         FOR i IN 1..v_ids_to_delete.COUNT LOOP
34             DELETE FROM lignecommandes1
35                 WHERE idlignecommande = v_ids_to_delete(i);
36         END LOOP;
37         pkg_synchro.is_replicating := FALSE;
38     END IF;
39 EXCEPTION WHEN OTHERS THEN
40     pkg_synchro.is_replicating := FALSE; RAISE;
41 END AFTER STATEMENT;
42 END SYNC_INSERT_LIGNE_SITE1;
43 /

```

4.5 Triggers BEFORE – Prévention des violations de clé étrangère

Lors d'un UPDATE modifiant idproduit, Oracle vérifie la clé étrangère **avant** les triggers AFTER. Un trigger BEFORE rapatrie le nouveau produit depuis le **Master** en amont du contrôle.

TRG_PULL_PARENTS_SITE1 – Rapatriement des entités parentes

```

1 CREATE OR REPLACE TRIGGER TRG_PULL_PARENTS_SITE1
2 BEFORE INSERT ON lignecommandes1 FOR EACH ROW
3 DECLARE
4     v_cnt NUMBER;
5     v_cmd commandes%ROWTYPE; v_cli clients%ROWTYPE;
6     v_prod produits%ROWTYPE;
7 BEGIN
8     IF pkg_synchro.is_replicating = TRUE THEN RETURN; END IF;
9     -- Rapatriement du produit
10    SELECT COUNT(*) INTO v_cnt FROM produits1 WHERE idproduit = :NEW.idproduit;
11    IF v_cnt = 0 THEN
12        SELECT * INTO v_prod FROM produits WHERE idproduit = :NEW.idproduit;
13        INSERT INTO produits1 VALUES v_prod;
14    END IF;
15    -- Rapatriement de la commande et du client
16    SELECT COUNT(*) INTO v_cnt FROM commandes1 WHERE idcommande = :NEW.
        idcommande;

```

```
17     IF v_cnt = 0 THEN
18         SELECT * INTO v_cmd FROM commandes WHERE idcommande = :NEW.idcommande;
19         SELECT COUNT(*) INTO v_cnt FROM clients1 WHERE idclient = v_cmd.
           idclient;
20     IF v_cnt = 0 THEN
21         SELECT * INTO v_cli FROM clients WHERE idclient = v_cmd.idclient;
22         INSERT INTO clients1 VALUES v_cli;
23     END IF;
24     INSERT INTO commandes1 VALUES v_cmd;
25 END IF;
26 END;
27 /
```

5. Scénario 2 – Fragmentation par Seuil de Quantité

5.1 Règles de fragmentation

$$R_1 = \sigma_{quantité \geq 100}(LigneCommandes) \longrightarrow \text{Site 1}$$

$$R_2 = \sigma_{quantité < 100}(LigneCommandes) \longrightarrow \text{Site 2}$$

$\{R_1, R_2\}$ forme une **partition complète** : toute ligne de commande appartient à exactement un fragment, sans exception.

5.2 Différences clés avec le Scénario 1

	Scénario 1	Scénario 2
Critère R_1	idcateg=50 ET qté>100	qté≥100
Critère R_2	idcateg=35 ET qté>50	qté<100
Partition	Partielle	Totale
Cat 10 (fournitures)	Master uniquement	Site 1 (qté≥100)
Cat 35 (mobilier)	Site 2 si qté>50	Réparti selon quantité
Routage trigger	2 critères	1 seul critère
Migration UP-DATE	Catégorie OU quantité change	Quantité franchit 100

5.3 Initialisation par CTAS

CTAS Scénario 2 – Site 1 et Site 2

```

1  -- Site 1 : R1 = toutes lignes avec quantite >= 100 (toutes categories)
2  CREATE TABLE lignecommandes1 AS
3      SELECT lc.idlignecommande, lc.idcommande, lc.idproduit,
4             lc.quantite, lc.remise
5      FROM   lignecommandes@link_central lc
6      WHERE  lc.quantite >= 100;    -- pas de filtre sur idcateg
7
8  -- Site 2 : R2 = toutes lignes avec quantite < 100
9  CREATE TABLE lignecommandes2 AS
10     SELECT lc.idlignecommande, lc.idcommande, lc.idproduit,
11            lc.quantite, lc.remise
12     FROM   lignecommandes@link_central lc
13     WHERE  lc.quantite < 100;

```

5.4 Triggers de distribution – Master (Scénario 2)

Le routage est simplifié : un seul critère (la quantité) suffit. L'UPDATE gère 4 cas selon le franchissement du seuil 100.

SYC_INSERT_LIGNE et SYC_UPDATE_LIGNE – Master (Scénario 2)

```

1  -- INSERT : routage par seuil de quantite
2  CREATE OR REPLACE TRIGGER SYC_INSERT_LIGNE
3  AFTER INSERT ON lignecommandes FOR EACH ROW
4  DECLARE
5      NQ lignecommandes.quantite%TYPE := :NEW.quantite;
6  BEGIN
7      IF pkg_synchro.is_replicating = TRUE THEN RETURN; END IF;
8      IF NQ >= 100 THEN                                -- R1 -> Site 1
9          EXECUTE IMMEDIATE
10             'BEGIN INSERTligne@link_site1(:1,:2,:3,:4,:5); END; '
11             USING :NEW.idlignecommande, :NEW.idcommande,
12                 :NEW.idproduit, :NEW.quantite, :NEW.remise;
13      ELSE                                              -- R2 -> Site 2
14          EXECUTE IMMEDIATE
15             'BEGIN INSERTligne@link_site2(:1,:2,:3,:4,:5); END; '
16             USING :NEW.idlignecommande, :NEW.idcommande,
17                 :NEW.idproduit, :NEW.quantite, :NEW.remise;
18      END IF;
19  END;
20  /
21
22  -- UPDATE : 4 cas de migration
23  CREATE OR REPLACE TRIGGER SYC_UPDATE_LIGNE
24  AFTER UPDATE ON lignecommandes FOR EACH ROW
25  DECLARE
26      OQ lignecommandes.quantite%TYPE := :OLD.quantite;

```

```

27     NQ lignecommandes.quantite%TYPE := :NEW.quantite;
28 BEGIN
29     IF pkg_synchro.is_replicating = TRUE THEN RETURN; END IF;
30
31     IF OQ >= 100 AND NQ >= 100 THEN          -- reste sur Site 1
32         EXECUTE IMMEDIATE
33             'BEGIN updateligne@link_site1(:1,:2,:3,:4); END;';
34         USING :NEW.idlignecommande,:NEW.idproduit,:NEW.quantite,:NEW.remise;
35
36     ELSIF OQ >= 100 AND NQ < 100 THEN          -- Site 1 --> Site 2
37         EXECUTE IMMEDIATE 'BEGIN DELETEligne@link_site1(:1); END;';
38         USING :OLD.idlignecommande;
39         EXECUTE IMMEDIATE
40             'BEGIN INSERTligne@link_site2(:1,:2,:3,:4,:5); END;';
41         USING :NEW.idlignecommande,:NEW.idcommande,
42             :NEW.idproduit,:NEW.quantite,:NEW.remise;
43
44     ELSIF OQ < 100 AND NQ >= 100 THEN          -- Site 2 --> Site 1
45         EXECUTE IMMEDIATE 'BEGIN DELETEligne@link_site2(:1); END;';
46         USING :OLD.idlignecommande;
47         EXECUTE IMMEDIATE
48             'BEGIN INSERTligne@link_site1(:1,:2,:3,:4,:5); END;';
49         USING :NEW.idlignecommande,:NEW.idcommande,
50             :NEW.idproduit,:NEW.quantite,:NEW.remise;
51
52     ELSE                                         -- reste sur Site 2
53         EXECUTE IMMEDIATE
54             'BEGIN updateligne@link_site2(:1,:2,:3,:4); END;';
55         USING :NEW.idlignecommande,:NEW.idproduit,:NEW.quantite,:NEW.remise;
56     END IF;
57 END;
58 /

```

5.5 Triggers compound sur Site 2 (Scénario 2)

Sur Site 2, le trigger compound détecte si la nouvelle quantité franchit le seuil vers le haut (≥ 100) et migre la ligne vers Site 1.

SYC_UPDATE_LIGNE_SITE2 – Migration automatique vers Site 1

```

1 CREATE OR REPLACE TRIGGER SYC_UPDATE_LIGNE_SITE2
2 FOR UPDATE ON lignecommandes2
3 COMPOUND TRIGGER
4     TYPE t_id_list IS TABLE OF lignecommandes2.idlignecommande%TYPE;
5     v_ids_to_delete t_id_list := t_id_list();
6
7     AFTER EACH ROW IS
8     BEGIN
9         IF pkg_synchro.is_replicating = FALSE OR
10            pkg_synchro.is_replicating IS NULL THEN
11             -- 1. Propager la mise a jour au Master
12             EXECUTE IMMEDIATE
13                 'BEGIN updateligne@link_central(:1,:2,:3,:4); END;';

```

```
14      USING :NEW.idlignecommande,:NEW.idproduit,
15            :NEW.quantite,:NEW.remise;
16
17      -- 2. Si la ligne franchit le seuil (< 100 -> >= 100)
18      IF :NEW.quantite >= 100 THEN
19          EXECUTE IMMEDIATE
20              'BEGIN INSERTligne@link_site1(:1,:2,:3,:4,:5); END;';
21          USING :NEW.idlignecommande,:NEW.idcommande,
22                :NEW.idproduit,:NEW.quantite,:NEW.remise;
23          v_ids_to_delete.EXTEND;
24          v_ids_to_delete(v_ids_to_delete.LAST) := :OLD.idlignecommande;
25      END IF;
26  END IF;
27  END AFTER EACH ROW;
28
29  AFTER STATEMENT IS
30  BEGIN
31      IF v_ids_to_delete.COUNT > 0 THEN
32          pkg_synchro.is_replicating := TRUE;
33          FOR i IN 1..v_ids_to_delete.COUNT LOOP
34              DELETE FROM lignecommandes2
35                  WHERE idlignecommande = v_ids_to_delete(i);
36          END LOOP;
37          pkg_synchro.is_replicating := FALSE;
38      END IF;
39  END AFTER STATEMENT;
40  END SYNC_UPDATE_LIGNE_SITE2;
41  /
```

6. Mécanismes de Synchronisation Communs

6.1 Package anti-boucle pkg_synchro

Principe du mécanisme anti-boucle

Sans protection, une écriture sur le **Master** déclenche un trigger vers Site 1, qui à son tour déclenche un trigger vers le **Master**, créant une boucle infinie. Le package `pkg_synchro` casse cette boucle en maintenant un drapeau de session.

Package pkg_synchro – identique sur les 3 nœuds

```
1 CREATE OR REPLACE PACKAGE pkg_synchro AS
2     is_replicating BOOLEAN := FALSE;
3 END pkg_synchro;
4 /
5 -- Utilisation dans les triggers :
6 -- IF pkg_synchro.is_replicating = TRUE THEN RETURN; END IF;
7 -- Et dans les procédures :
8 -- pkg_synchro.is_replicating := TRUE;
9 -- ... operation locale ...
10 -- pkg_synchro.is_replicating := FALSE;
```

6.2 Journalisation autonome des erreurs

Table sync_errors et procédure log_sync_error

```
1 CREATE TABLE sync_errors (
2     id_err    NUMBER PRIMARY KEY,
3     date_err  TIMESTAMP DEFAULT SYSTIMESTAMP,
4     contexte  VARCHAR2(100),
5     id_ligne  NUMBER,
6     message   VARCHAR2(4000)
7 );
8
9 -- PRAGMA AUTONOMOUS_TRANSACTION : l'erreur est persistee meme
10 -- si la transaction principale effectue un ROLLBACK.
11 CREATE OR REPLACE PROCEDURE log_sync_error(
12     p_ctx IN VARCHAR2, p_idl IN NUMBER, p_msg IN VARCHAR2
13 ) IS
14     PRAGMA AUTONOMOUS_TRANSACTION;
15 BEGIN
16     INSERT INTO sync_errors (contexte, id_ligne, message)
17     VALUES (p_ctx, p_idl, SUBSTR(p_msg, 1, 4000));
18     COMMIT;
```

```

19 END log_sync_error;
20 /

```

6.3 Procédures stockées PL/SQL

Les procédures INSERTligne, DELETEligne et updateligne sont créées sur chaque nœud. Elles encapsulent le drapeau anti-boucle et le rapatriement automatique des entités parentes depuis le Master.

INSERTligne – Site 1 (rapatriement automatique des parents)

```

1 CREATE OR REPLACE PROCEDURE INSERTligne(
2     p_idl NUMBER, p_idc NUMBER, p_idp NUMBER,
3     p_qte NUMBER, p_rem NUMBER
4 ) AS
5     v_nc INTEGER; v_cmd commandes1%ROWTYPE;
6     v_prod produits1%ROWTYPE; v_cli clients1%ROWTYPE;
7 BEGIN
8     pkg_synchro.is_replicating := TRUE;
9
10    -- Rapatriement commande + client depuis le Master si absents localement
11    SELECT COUNT(*) INTO v_nc FROM commandes1 WHERE idcommande = p_idc;
12    IF v_nc = 0 THEN
13        SELECT * INTO v_cmd FROM commandes WHERE idcommande = p_idc;
14        SELECT COUNT(*) INTO v_nc FROM clients1 WHERE idclient = v_cmd.idclient
15        ;
16        IF v_nc = 0 THEN
17            SELECT * INTO v_cli FROM clients WHERE idclient = v_cmd.idclient;
18            INSERT INTO clients1 VALUES v_cli;
19        END IF;
20        INSERT INTO commandes1 VALUES v_cmd;
21    END IF;
22
23    -- Rapatriement produit depuis le Master si absent
24    SELECT COUNT(*) INTO v_nc FROM produits1 WHERE idproduit = p_idp;
25    IF v_nc = 0 THEN
26        SELECT * INTO v_prod FROM produits WHERE idproduit = p_idp;
27        INSERT INTO produits1 VALUES v_prod;
28    END IF;
29
30    INSERT INTO lignecommandes1
31        (idlignecommande,idcommande,idproduit,quantite,remise)
32    VALUES (p_idl, p_idc, p_idp, p_qte, p_rem);
33
34    pkg_synchro.is_replicating := FALSE;
35 EXCEPTION WHEN OTHERS THEN
36     pkg_synchro.is_replicating := FALSE;
37     RAISE_APPLICATION_ERROR(-20001, 'INSERTligne SITE1: '||SQLERRM);
38 END INSERTligne;
39 /

```


6.4 Sauvegarde par vues matérialisées

Vues matérialisées – Base de secours

```
1 CONNECT eshop_backup/EshopPassword123@//localhost:1521/XEPDB1
2
3 -- Recopie complete toutes les 5 minutes.
4 -- Aucun MV LOG requis cote Master (zero impact production).
5 CREATE MATERIALIZED VIEW mv_lignecommandes
6     REFRESH COMPLETE START WITH SYSDATE NEXT SYSDATE + 5/1440
7     AS SELECT * FROM lignecommandes@link_central;
8
9 -- Idem pour clients, produits, commandes
```

7. Optimisation des Requêtes

7.1 Q5 – Nombre de commandes par client (analyse du plan)

Q5 – Requête et index couvrant

```
1  -- Requete metier
2  SELECT cl.codeclient, cl.societe, COUNT(*) AS nb_commandes_2026
3  FROM    commandes c
4  JOIN    clients    cl ON cl.idclient = c.idclient
5  WHERE   c.datecommande >= DATE '2026-01-01'
6  AND     c.datecommande <  DATE '2027-01-01'
7  GROUP BY cl.idclient, cl.codeclient, cl.societe
8  ORDER BY nb_commandes_2026 DESC;
9
10 -- Index couvrant : sert le filtre de plage ET fournit idclient
11 -- sans acceder a la table (index-only scan possible)
12 CREATE INDEX ix_cmd_date_client ON commandes(datecommande, idclient);
```

Opération	Objet	Lignes	Coût	État
TABLE ACCESS FULL	COMMANDES	100	3	Avant index
TABLE ACCESS FULL	CLIENTS	50	2	Avant index
INDEX RANGE SCAN	IX_CMD_DATE_CLIENT	100	1	Après index
TABLE ACCESS FULL	CLIENTS	50	2	Après index

Gain mesuré

Remplacement du TABLE ACCESS FULL (coût 3) par INDEX RANGE SCAN (coût 1) sur COMMANDES. Coût total : 7 → **5** (−28%). Gain exponentiel sur grand volume de données.

7.2 Q6 – Chiffre d'affaires distribué (UNION ALL)

Q6 – CA par catégorie via DB Links

```

1  -- Execution depuis le Master
2  ALTER SESSION SET CURRENT_SCHEMA = eshop_admin;
3
4  SELECT idcateg, ROUND(SUM(ca_ligne), 2) AS chiffre_affaires_2026
5  FROM (
6      -- Contribution Site 1 (R1)
7      SELECT p.idcateg,
8             lc.quantite * p.prixunitaire * (1 - NVL(lc.remise,0)) AS ca_ligne
9      FROM   lignecommandes1 lc    -- synonyme public -> @link_site1
10     JOIN   produits1 p ON p.idproduit = lc.idproduit
11     JOIN   commandes1 c ON c.idcommande = lc.idcommande
12     WHERE  c.datecommande BETWEEN DATE '2026-01-01' AND DATE '2026-12-31'
13
14     UNION ALL -- fragments disjoints : UNION ALL (pas de doublons)
15
16     -- Contribution Site 2 (R2)
17     SELECT p.idcateg,
18            lc.quantite * p.prixunitaire * (1 - NVL(lc.remise,0)) AS ca_ligne
19     FROM   lignecommandes2 lc    -- synonyme public -> @link_site2
20     JOIN   produits2 p ON p.idproduit = lc.idproduit
21     JOIN   commandes2 c ON c.idcommande = lc.idcommande
22     WHERE  c.datecommande BETWEEN DATE '2026-01-01' AND DATE '2026-12-31'
23 )
24 GROUP BY idcateg ORDER BY chiffre_affaires_2026 DESC;

```

Pourquoi UNION ALL et non UNION ?

R_1 et R_2 sont disjoints par construction : un même `idlignecommande` ne peut exister que dans un seul fragment. UNION effectuerait un tri inutile pour éliminer des doublons inexistantes. UNION ALL est donc à la fois correct et plus performant.

8. Tests et Résultats

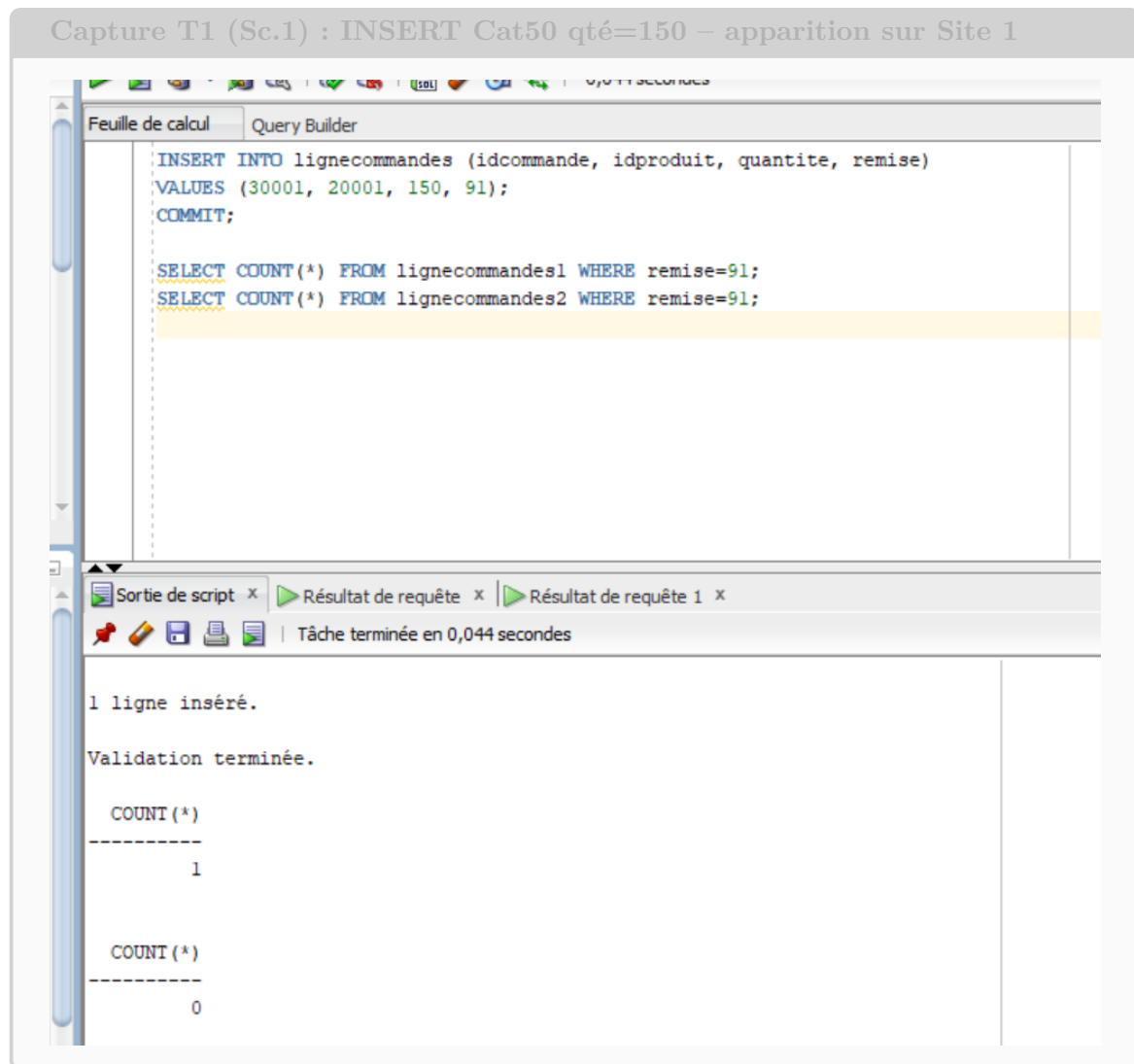
8.1 Méthodologie de test

Les tests valident trois aspects :

1. **Propagation initiale** : INSERT sur le Master → apparition sur le bon site.
2. **Migration par UPDATE** : modification d'un attribut de routage → déplacement entre fragments.
3. **Modification inter-sites** : UPDATE exécuté depuis Site 2 sur un élément appartenant à Site 1.

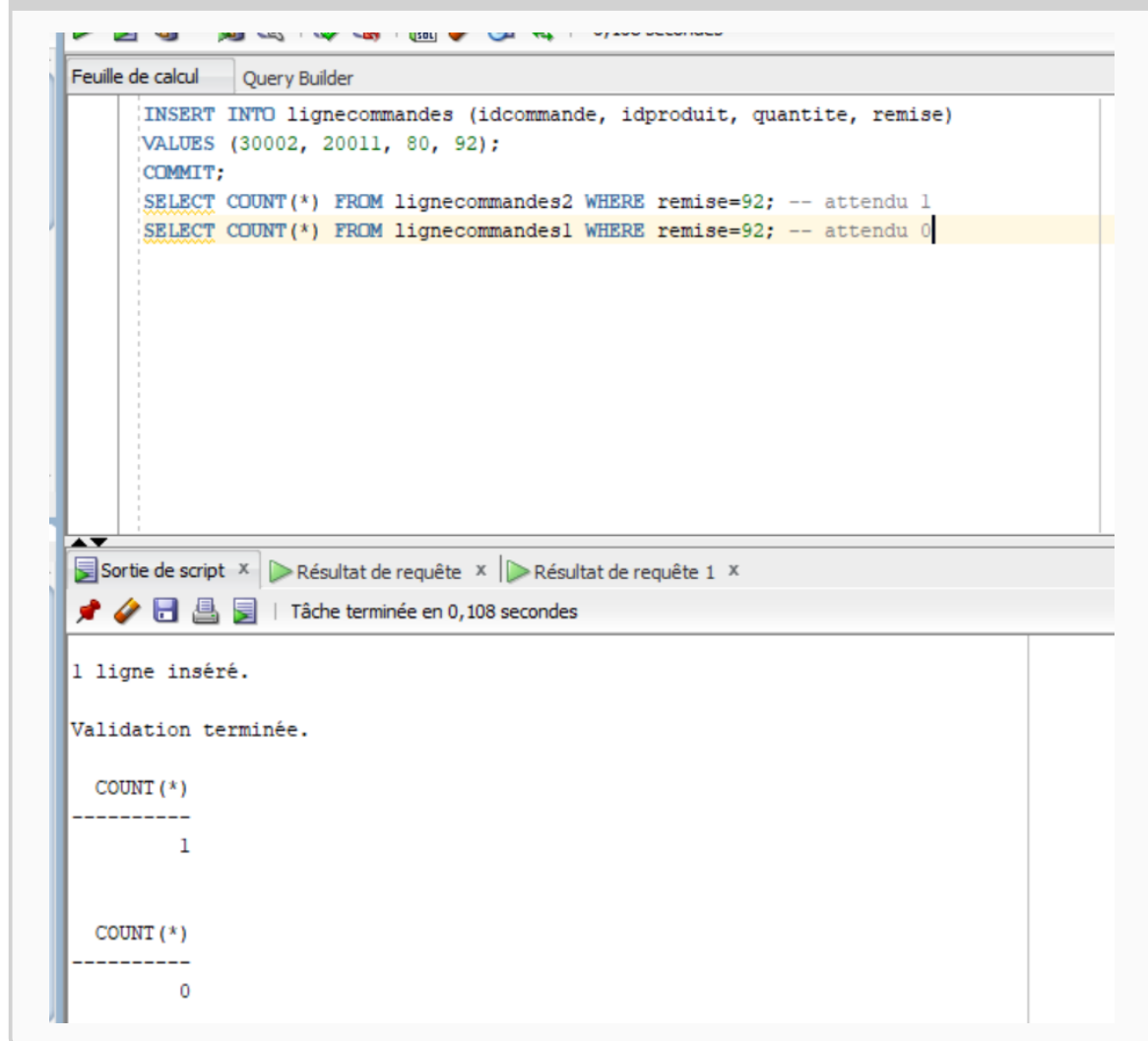
8.2 Tests – Scénario 1

8.2.1 T1 – INSERT Cat50, qté=150 → Site 1

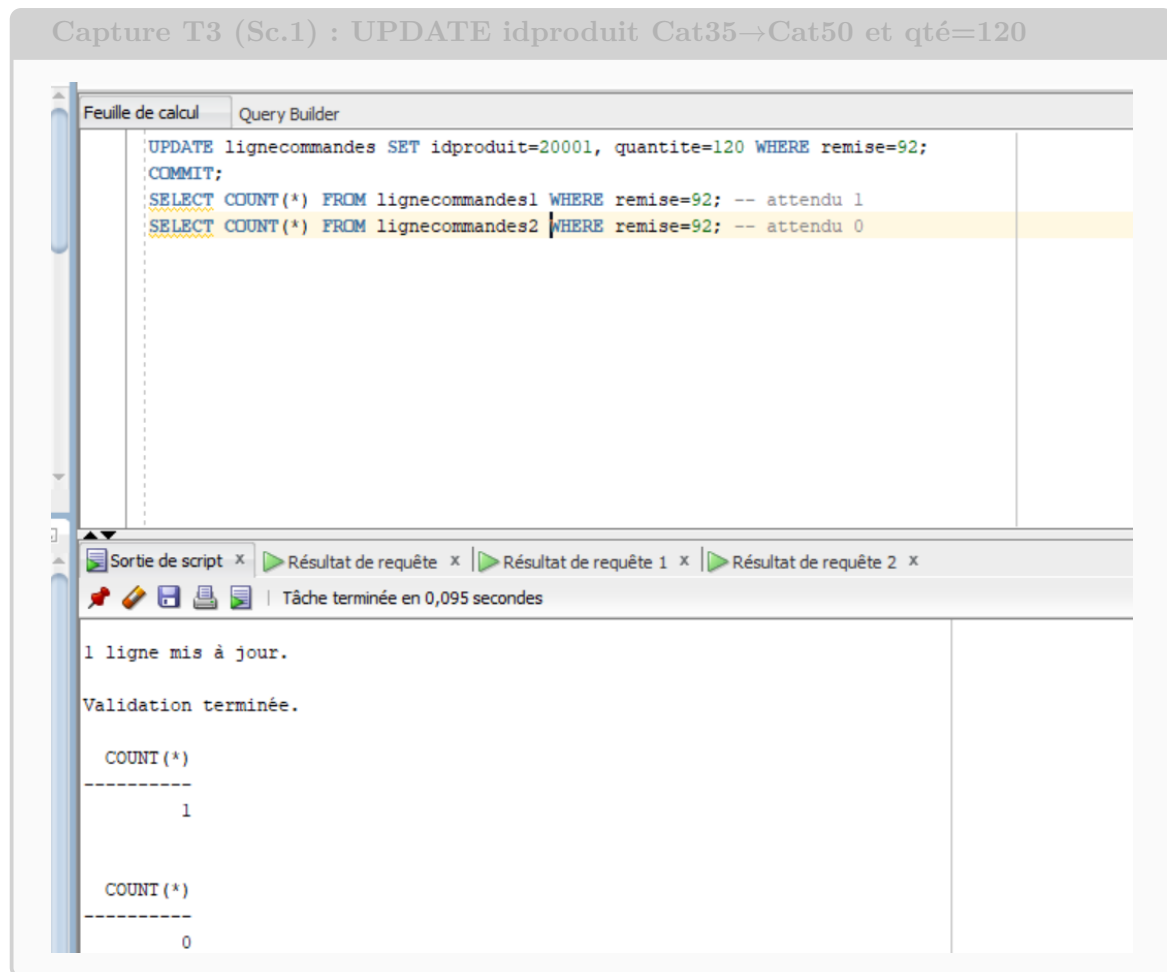


8.2.2 T2 – INSERT Cat35, qté=80 → Site 2

Capture T2 (Sc.1) : INSERT Cat35 qté=80 – apparition sur Site 2

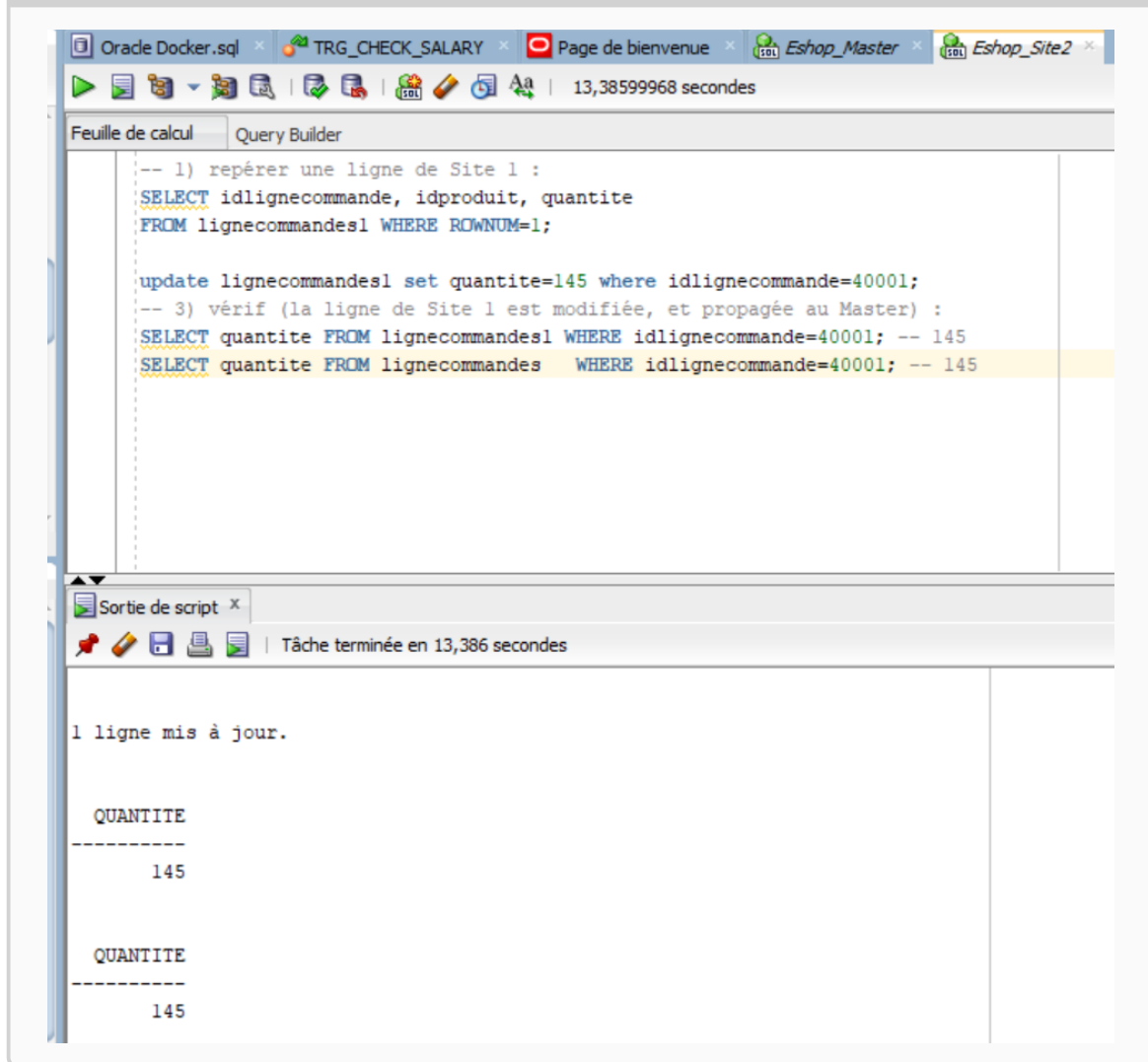


8.2.3 T3 – UPDATE : migration Site 2 → Site 1 (changement de catégorie)

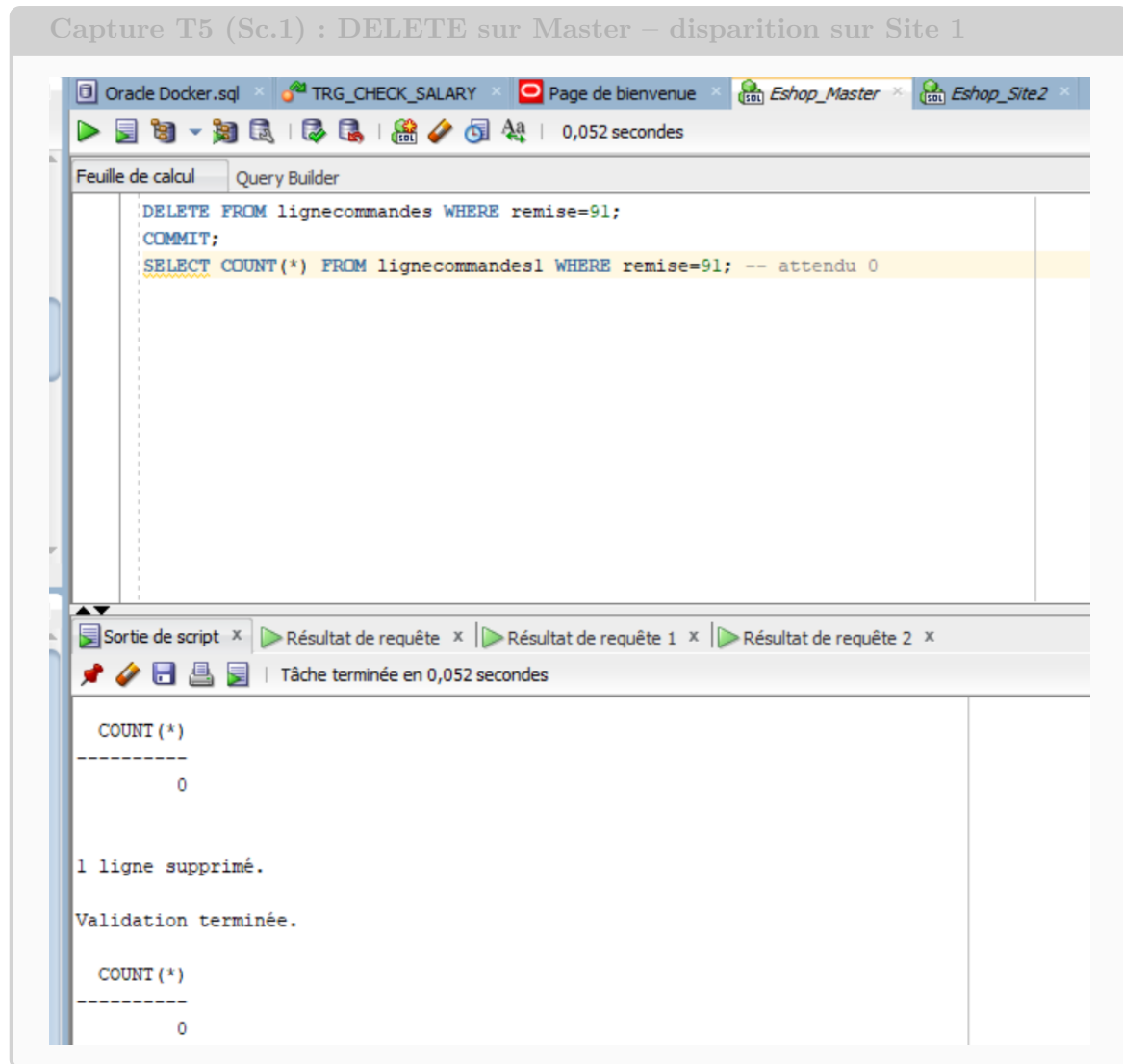


8.2.4 T4 – Modification depuis Site 2 vers Site 1 (inter-sites)

Capture T4 (Sc.1) : UPDATE exécuté depuis Site 2 sur un élément de Site 1



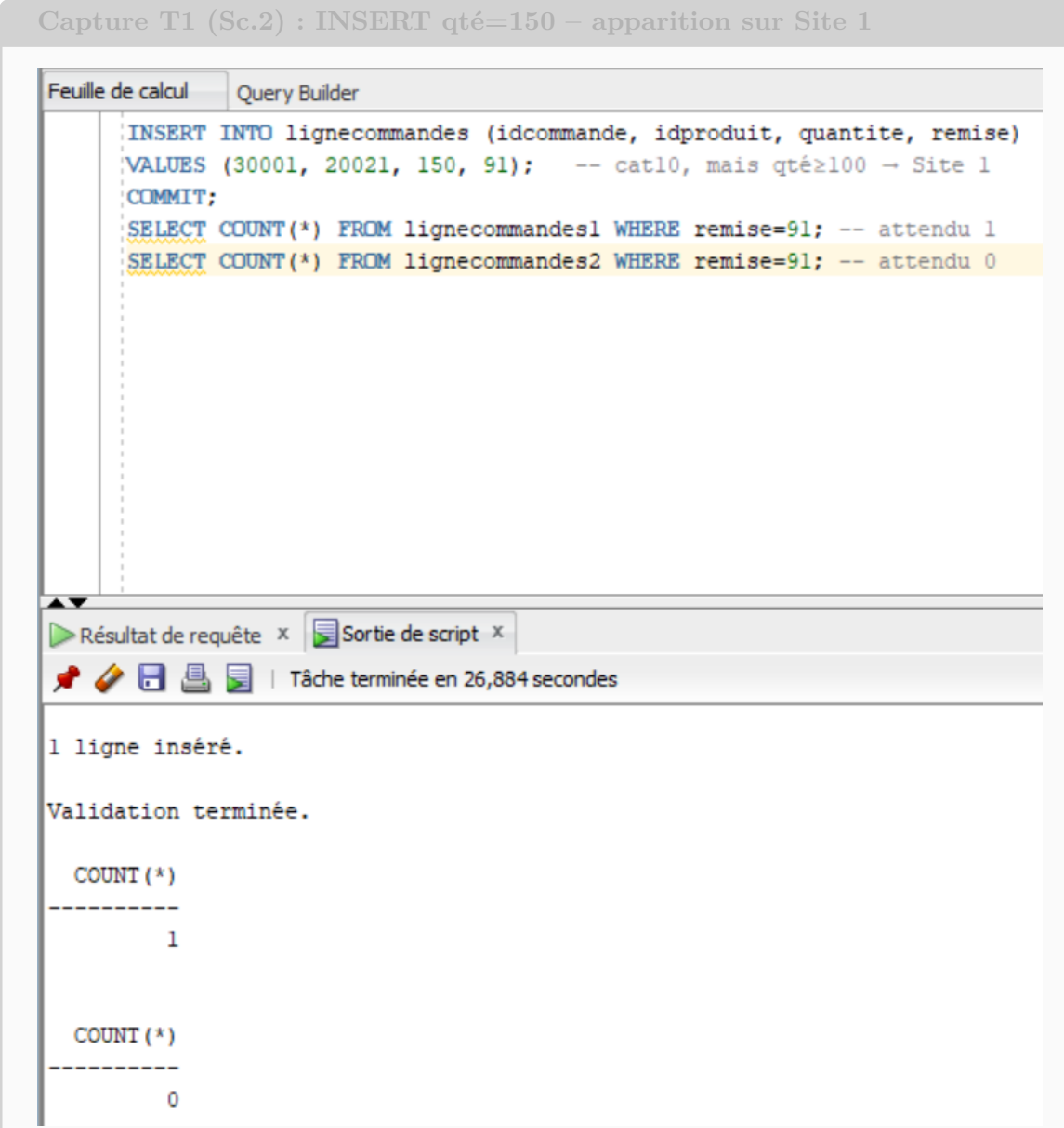
8.2.5 T5 – Propagation de la suppression (DELETE)



8.3 Tests – Scénario 2

8.3.1 T1 – INSERT qté=150 → Site 1 (toute catégorie)

Capture T1 (Sc.2) : INSERT qté=150 – apparition sur Site 1



The screenshot shows a database query tool interface. At the top, there's a title bar 'Capture T1 (Sc.2) : INSERT qté=150 – apparition sur Site 1'. Below it, a tabbed window with 'Feuille de calcul' and 'Query Builder'. The 'Query Builder' tab is active, showing an SQL script:

```
INSERT INTO lignecommandes (idcommande, idproduit, quantite, remise)
VALUES (30001, 20021, 150, 91); -- cat10, mais qté≥100 → Site 1
COMMIT;
SELECT COUNT(*) FROM lignecommandes1 WHERE remise=91; -- attendu 1
SELECT COUNT(*) FROM lignecommandes2 WHERE remise=91; -- attendu 0
```

Below the script, there's a toolbar with icons for running queries, saving, and printing. The status bar indicates 'Tâche terminée en 26,884 secondes'. The results pane shows the output of the queries:

1 ligne inséré.

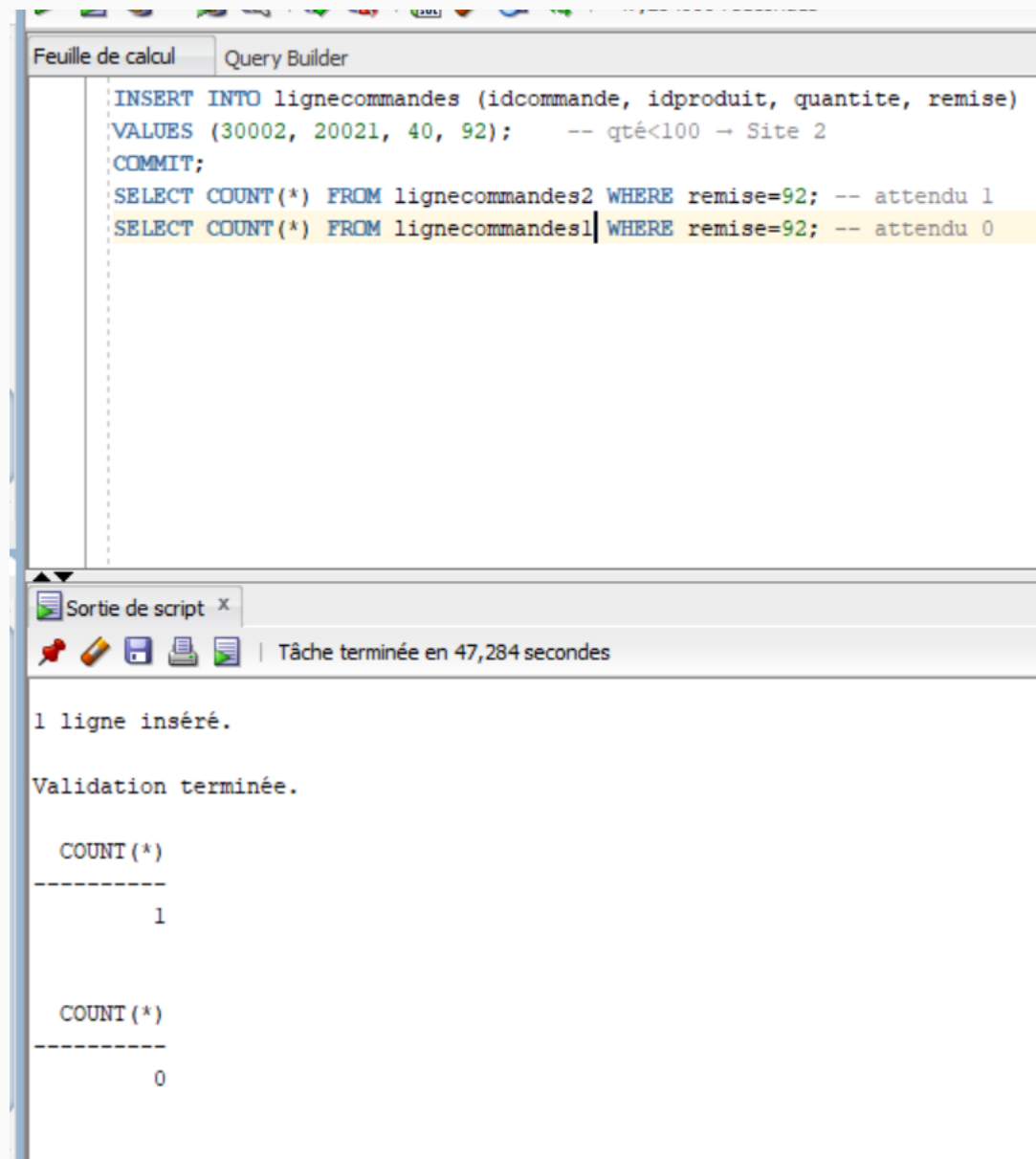
Validation terminée.

COUNT (*)
1

COUNT (*)
0

8.3.2 T2 – INSERT qté=40 → Site 2

Capture T2 (Sc.2) : INSERT qté=40 – apparition sur Site 2



8.3.3 T3 – UPDATE depuis Site 2 : migration Site 2 → Site 1

Capture T3 (Sc.2) : UPDATE qté 40→150 depuis Site 2 – migration vers Site 1

Oracle Docker.sql | TRG_CHECK_SALARY | Page de bienvenue | Eshop_Master | Eshop_Site2 | Master | Site 2 | 27,15299988 secondes

Feuille de calcul | Query Builder

```
-- repérer l'id de la ligne (remise=92) sur Site 2 :
SELECT idlignecommande FROM lignecommandes2 WHERE remise=92;
UPDATE lignecommandes2 SET quantite=150 WHERE idlignecommande=40303;
COMMIT;
SELECT COUNT(*) FROM lignecommandes2 WHERE idlignecommande=40303; -- 0 (migrée)
SELECT quantite FROM lignecommandes1 WHERE idlignecommande=40303; -- 150
SELECT quantite FROM lignecommandes WHERE idlignecommande=40303; -- 150
```

Sortie de script | Résultat de requête | Tâche terminée en 27,153 secondes

1 ligne mis à jour.

Validation terminée.

COUNT (*)	0
QUANTITE	150
QUANTITE	150

8.3.4 T4 – UPDATE : migration Site 1 → Site 2

Capture T4 (Sc.2) : UPDATE qté 150→30 – retour vers Site 2

The screenshot shows the Oracle SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database functions. The main window displays a SQL script with the following content:

```
UPDATE lignecommandes SET quantite=30 WHERE remise=92;
COMMIT;
SELECT COUNT(*) FROM lignecommandes1 WHERE remise=92; -- attendu 0
SELECT COUNT(*) FROM lignecommandes2 WHERE remise=92; -- attendu 1
```

Below the script, the 'Sortie de script' (Script Output) window shows the execution results:

```
1 ligne mis à jour.
Validation terminée.

COUNT (*)
-----
0

COUNT (*)
-----
1
```

8.3.5 T5 – Vérification de la partition complète

Capture T5 (Sc.2) : vérification de l'absence de doublons entre fragments

The screenshot shows the Oracle SQL Developer interface. The main window displays a SQL script with the following content:

```
SELECT COUNT(*) FROM (
  SELECT idlignecommande FROM lignecommandes1
  INTERSECT
  SELECT idlignecommande FROM lignecommandes2
); -- attendu 0
```

Below the script, the 'Sortie de script' (Script Output) window shows the execution results:

```
COUNT (*)
-----
0
```

8.4 Q5 – Plan d'exécution avant et après index

Capture Q5 : EXPLAIN PLAN avant l'index – TABLE ACCESS FULL

```
=====
AVANT INDEX : plan d'execution
=====

Explained.

PLAN_TABLE_OUTPUT
-----
Plan hash value: 2791779646

-----
| Id | Operation          | Name      | Rows  | Cost (%CPU)|
-----
|  0 | SELECT STATEMENT   |           |    100 |       7 (15)|
|  1 |   HASH GROUP BY    |           |    100 |       7 (15)|
|  2 |     HASH JOIN       |           |    100 |       6 ( 0)|
|  3 |       TABLE ACCESS FULL | CLIENTS  |     50 |       3 ( 0)|
|  4 |       TABLE ACCESS FULL | COMMANDES |    100 |       3 ( 0)|
-----

11 rows selected.
```

Capture Q5 : EXPLAIN PLAN après l'index – INDEX RANGE SCAN

```
=====
APRES INDEX : plan d'execution
=====

Explained.

PLAN_TABLE_OUTPUT
-----
Plan hash value: 4089004230

-----
| Id | Operation          | Name           | Rows  | Cost (%CPU)|
-----
|  0 | SELECT STATEMENT   |               |     50 |       5 (20)|
|  1 |   HASH JOIN        |               |     50 |       5 (20)|
|  2 |     VIEW            | VW_GBF_7       |     50 |       2 (50)|
|  3 |       HASH GROUP BY |               |     50 |       2 (50)|
|  4 |         INDEX RANGE SCAN | IX_CMD_DATE_CLIENT |    100 |       1 ( 0)|
|  5 |           TABLE ACCESS FULL | CLIENTS       |     50 |       3 ( 0)|
-----

12 rows selected.
```

8.5 Q6 – Requête distribuée : CA par catégorie

Capture Q6 (Sc.1) : CA par catégorie via UNION ALL – Scénario 1

```
SQL> @/queries/Q6_distributed_query.sql

Session altered.

  IDCATEG  CHIFFRE_AFFAIRES_2026
-----
        50          204973845
        35          16328640.8

SQL> █
```

Capture Q6 (Sc.2) : CA par catégorie via UNION ALL – Scénario 2

```
SQL> @/queries/Q6_distributed_query.sql

Session altered.

  IDCATEG  CHIFFRE_AFFAIRES_2026  NB_LIGNES
-----
        50          204973845         100
        35          16328640.8         100
        10           5778690          100

SQL> █
```

Conclusion

Ce projet a permis de concevoir et d'implémenter une architecture complète de base de données distribuée sur Oracle XE 21c pour la plateforme B2B **EShop**. Les principaux apports sont :

- **Deux scénarios de fragmentation** opérationnels, chacun disposant de ses propres scripts d'initialisation, de synchronisation et de test, organisés dans des répertoires dédiés (`scripts/scenario1/` et `scripts/scenario2/`).
- **Séquences sans collision** : la technique du décalage modulo 3 garantit mathématiquement l'unicité des identifiants distribués, sans aucun mécanisme de verrouillage inter-nœuds.
- **Réplication multi-maître robuste** : le package `pkg_synchro` casse les boucles de synchronisation ; les triggers compound contournent ORA-04091 ; la table `sync_errors` trace toute anomalie sans bloquer les transactions.
- **Migration transparente** : un UPDATE qui franchit un seuil de routage déclenche automatiquement la migration de la ligne vers le bon fragment, sans intervention applicative.
- **Optimisation des requêtes** : l'index couvrant `ix_cmd_date_client` réduit le coût de Q5 de 28% ; la requête Q6 utilise UNION ALL sur les deux sites distants pour calculer le chiffre d'affaires consolidé.
- **Haute disponibilité** : la base de secours réplique l'intégralité du **Master** par vues matérialisées toutes les 5 minutes, sans impact sur la production.

Ce travail constitue une implémentation concrète des concepts fondamentaux des bases de données distribuées : fragmentation, réplication, cohérence distribuée et optimisation de requêtes fédérées.